

Análisis y Diseño con Patrones

PEC 2: Patrones de Diseño y de Asignación de Responsabilidades

En esta PEC supondremos que trabajamos como ingenieros de software en distintos proyectos del sector tecnológico. Este tipo de proyectos es habitual en la industria moderna, donde los sistemas de software deben resolver problemas complejos en dominios muy diversos: desde aplicaciones médicas o de ingeniería hasta herramientas de gestión y validación de datos.

La problemática que aborda el desarrollo de este tipo de sistemas es de una complejidad considerable. Abarca desde la gestión del estado y la coordinación entre componentes, hasta la integración con librerías externas y la estructuración de algoritmos reutilizables.

En nuestro caso, aplicaremos la abstracción para simplificar cada dominio. Nos centraremos en problemas aislados y bien definidos en cada ejercicio. Esto refleja el día a día de muchos ingenieros que trabajan en sistemas complejos.

 **NOTA:** Todos los modelos UML que se entreguen deben estar realizados mediante una herramienta de modelado. No se aceptan modelos realizados a mano, ni con herramientas de dibujo tipo *Paint*, ya que estos modelos no tienen rigor. Se recomienda usar la herramienta [VisualParadigm community edition](https://visualparadigm.com/).

 **FORMATO:** El documento que entregues con tu propuesta de solución para esta PEC **debe seguir el siguiente formato** (el texto en cursiva corresponde a información que debes rellenar):

Alumno: ...*tu nombre*...

Compromiso:

...*Certifico que*...

Pregunta 1.1:

...*tu propuesta de solución*...

...

 **NOTA:** Muchas de las preguntas de esta PEC piden información sobre posibles requisitos del sistema elegido. No se trata de dar respuestas que se ajusten 100% con la realidad ni con la plataforma mencionada, ya que no disponemos de información tan detallada sobre el sistema; sino de dar respuestas plausibles, que tengan sentido desde el punto de vista de lo que se explica en los materiales y que se ajusten a lo que conocemos del sistema. Se valorará especialmente la justificación a las decisiones tomadas, por encima de que se ajusten con exactitud a la realidad.

 **RECUERDA:** Tal como se explica en el Plan docente de la asignatura, esta actividad debe resolverse de forma individual. Por otro lado, no está permitido el uso de herramientas de inteligencia artificial. **En caso de detectar copias o uso de herramientas de IA, se penalizará la actividad con 0 % como nota.**

Para garantizar que has resuelto esta actividad de forma individual y sin uso de herramientas de IA, te pedimos que en tu solución entregada añadas el siguiente compromiso de autorresponsabilidad:

Certifico que he realizado la PEC2 de forma completamente individual y únicamente con la ayuda que el profesorado de esta asignatura considera oportuna, según las indicaciones explicadas en el documento "Originalidad en la evaluación" disponible en el aula. Entiendo que el trabajo no original y/o el empleo de IA generativa implicará que la actividad entregada no será corregida y que automáticamente se le asignará una calificación de 0 %.

Compromiso de autorresponsabilidad

Escribe aquí el texto con el compromiso de autorresponsabilidad que consta en cursiva en la nota previa. **No escribirlo implica la no corrección de la PEC y una calificación de 0 %.**

Certifico que he realizado la PEC2 de forma completamente individual y únicamente con la ayuda que el profesorado de esta asignatura considera oportuna, según las indicaciones explicadas en el documento "Originalidad en la evaluación" disponible en el aula. Entiendo que el trabajo no original y/o el empleo de IA generativa implicará que la actividad entregada no será corregida y que automáticamente se le asignará una calificación de 0 %.

Pregunta 1 (20 %)

En una red de computación neuronal distribuida, existe un componente crítico denominado *GlobalStateCoordinator*. Por razones de coherencia sináptica y para evitar conflictos de memoria, el sistema requiere que exista un único punto de acceso y control para la gestión de la configuración de red. Si dos coordinadores intentan gestionar la red simultáneamente, la estructura de datos se corrompería. Por tanto, el sistema debe garantizar que cualquier módulo que necesite consultar o modificar la configuración lo haga a través de un mismo punto en todo el ciclo de vida de la ejecución.

Por otro lado, la red está compuesta por múltiples *ProcessingNode*. Estos nodos necesitan ajustar su frecuencia de cálculo inmediatamente cada vez que la configuración en el coordinador cambia (p. ej., ante un aumento de temperatura o carga de trabajo).

Sin embargo, el sistema debe ser eficiente:

- Los nodos no deben preguntar constantemente al coordinador si hay cambios.
- El coordinador mantiene una lista de los nodos activos.
- Cada nodo está especializado en un "Tipo de Tarea" (p. ej., Vision, Audio). Cuando el coordinador emite una actualización, esta incluye una "Etiqueta de Tarea". El coordinador debe recorrer sus nodos registrados y notificar únicamente a aquellos cuya especialización coincida con la etiqueta de la actualización, evitando interrumpir el trabajo de los nodos no afectados.

Se pide:

Pregunta 1.1 (5 %). ¿Qué patrón o patrones usarías para modelar este problema?

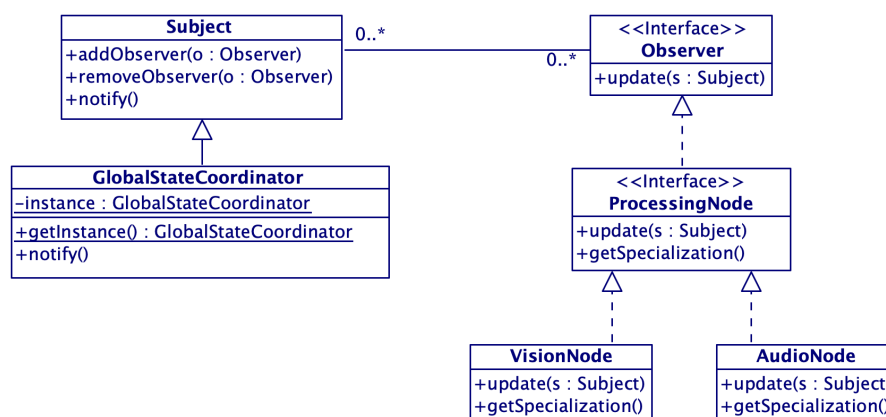
Pregunta 1.2 (15 %). Muestra el diagrama de clases UML que ilustra tu solución.

Solución

Pregunta 1.1. Por un lado, se tiene que emplear el patrón **Singleton** en el *GlobalStateCoordinator*. El objetivo es garantizar la unicidad de la instancia para mantener la coherencia sináptica y evitar que existan varios coordinadores compitiendo entre sí, lo que corrompería los datos de configuración. Al privatizar el constructor y centralizar el acceso a través de un método estático, se asegura que todos los nodos trabajen siempre sobre la misma fuente de verdad.

Por otro lado, se utiliza el patrón **Observer** para gestionar la relación entre el coordinador y los nodos. Con este patrón, es el propio coordinador quien les notifica los cambios de forma proactiva. Además, al integrar un filtrado por especialización, se garantiza que la notificación sea correcta y solamente interrumpa el trabajo de los nodos que realmente necesitan actualizarse.

Pregunta 1.2. El diagrama de clases sería el siguiente:



Pregunta 2 (20 %)

La corporación NeuralLink está desarrollando un dispositivo implantable que estabiliza la actividad cerebral en tiempo real. El sistema debe ser capaz de reaccionar de forma distinta según el nivel de alerta del usuario, pero manteniendo los algoritmos de procesamiento físico totalmente independientes de la lógica de control.

El dispositivo opera en tres modalidades distintas: *Safe*, *Active* e *Intensive*.. Cada modalidad decide cómo actuar. No solo se limitan a indicar el modo, sino que aplican políticas de seguridad sobre los datos. Por ejemplo, el modo *Safe* limitará cualquier resultado por encima de un umbral para proteger al paciente, mientras que el modo *Intensive* aplicará multiplicadores de potencia ante una crisis. Además, cada modalidad es responsable de monitorizar la estabilidad: si el modo *Active* detecta que la señal es demasiado errática, él mismo debe ordenar al regulador pasar a modo *Intensive*.

Independientemente de la modalidad de operación, la operación técnica de estabilizar el impulso depende de un algoritmo de filtrado. Este algoritmo es una pieza de lógica pura que puede variar según el modelo del implante o la necesidad del paciente (p. ej., Algoritmo Lineal, Algoritmo Gaussiano, etc.). Se requiere que estos algoritmos sean intercambiables y que el sistema pueda usar uno u otro sin tener que modificar el código de las modalidades de funcionamiento. Así, una misma modalidad (p. ej., *Active*) podría realizar sus tareas usando un algoritmo lineal en un paciente y uno gaussiano en otro.

Se pide:

Pregunta 2.1 (5 %). Identifica qué patrones permiten gestionar las casuísticas de este problema. En caso de que haya más de un patrón, explica cómo interactúan entre sí.

Pregunta 2.2 (15 %). Realiza el diagrama de clases que ilustra este problema.

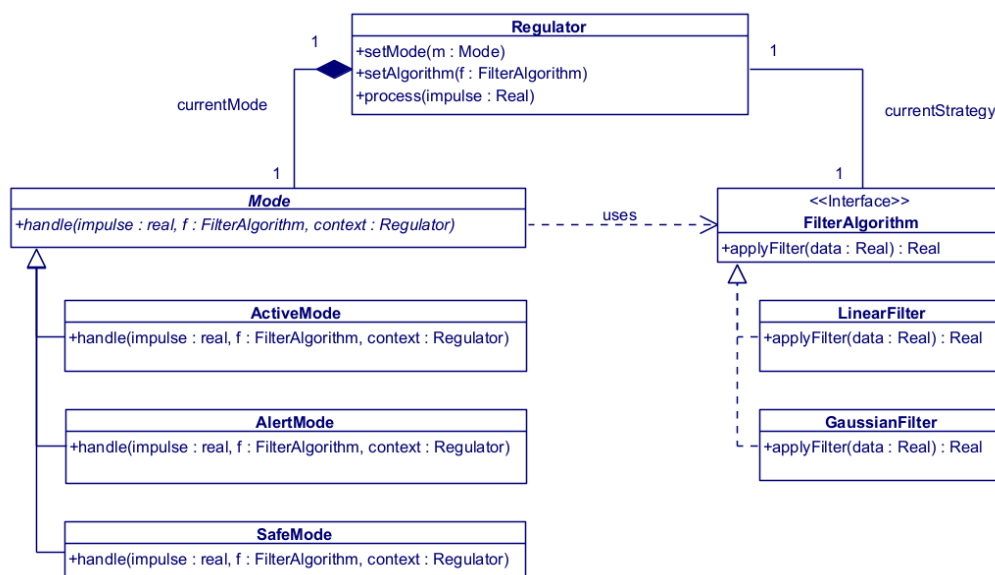
Solución

Pregunta 2.1. Por un lado, la gestión de las modalidades *Safe*, *Active* e *Intensive* se resuelve mediante el patrón **State**. En este contexto, el elemento *Regulator* actuará como el contexto que delega su comportamiento en un objeto. Al intercambiar este objeto, el regulador cambia su lógica de respuesta de forma transparente. De este modo, cada estado manejará el *handle* de un modo distinto, ya que cada estado debe comportarse de un modo distinto. Por ello, se le pasa el contexto del *Regulator* como parámetro, para que pueda transicionarlo, por ejemplo, en caso de que sea necesario.

También tenemos que utilizar el patrón **Strategy** para el manejo de los algoritmos de filtrado. Al modelar estos cálculos como estrategias intercambiables bajo una interfaz común, permitimos que las implementaciones concretas, ya sean lineales o gaussianas, puedan variar sin afectar al resto del sistema.

Cuando el regulador recibe un impulso, el estado activo invoca la estrategia de cálculo asignada para obtener el resultado técnico y, sobre esa base, aplica las reglas de control específicas de su propia modalidad.

Pregunta 2.2. El diagrama de clases es el siguiente:



Pregunta 3 (30 %)

La institución Archivo Digital utiliza un sistema para asegurar que los documentos escaneados no han sido alterados. El núcleo del sistema es un controlador llamado *DocumentValidator*, que se encarga de procesar las solicitudes de archivo. Para proteger el sistema de cambios tecnológicos, el controlador utiliza una interfaz propia llamada *IntegrityChecker* con un único método *isValid(Document d): Boolean*.

Recientemente, se ha adquirido una licencia de una herramienta forense avanzada llamada *UltraScannerSDK*. Sin embargo, esta librería externa es incompatible con nuestra interfaz. Para utilizarla, el flujo de trabajo es el siguiente:

1. Cargar el archivo con el método *initScan(File f)*.
2. Ejecutar el análisis mediante *performForensicAnalysis()*.
3. Extraer dos valores: *integrityScore* (un valor decimal entre 0 y 1) y *reliabilityIndex* (un porcentaje).

La dirección técnica indica que un documento solo es válido si el *integrityScore* es superior a 0.8 y el *reliabilityIndex* es mayor al 90%,

Una vez obtenida la respuesta del análisis, el controlador debe finalizar el proceso: si el documento es válido, se debe invocar el método *markAsValidated()* del documento para permitir su archivo definitivo; si el análisis falla, se debe invocar *markAsRejected()* para enviarlo a revisión manual.

Se pide:

Pregunta 3.1 (2,5 %). Indica qué patrón o patrones de diseño deben aplicarse para integrar *UltraScannerSDK* en el sistema

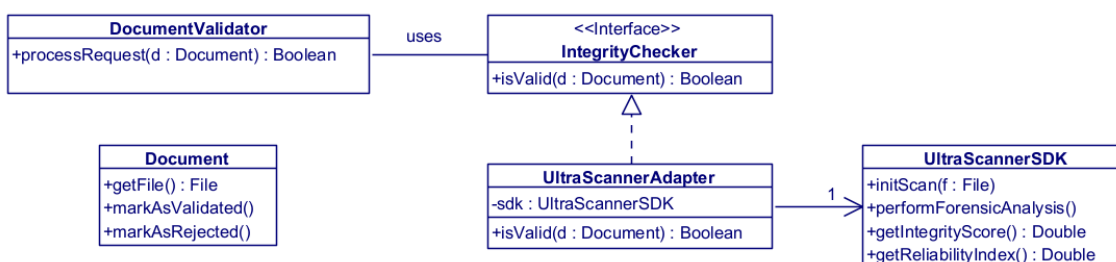
Pregunta 3.2 (12,5 %). Muestra el diagrama de clases UML con la integración de la nueva librería

Pregunta 3.3 (15 %). Realiza el diagrama de secuencia de la operación de validación utilizando la herramienta externa.

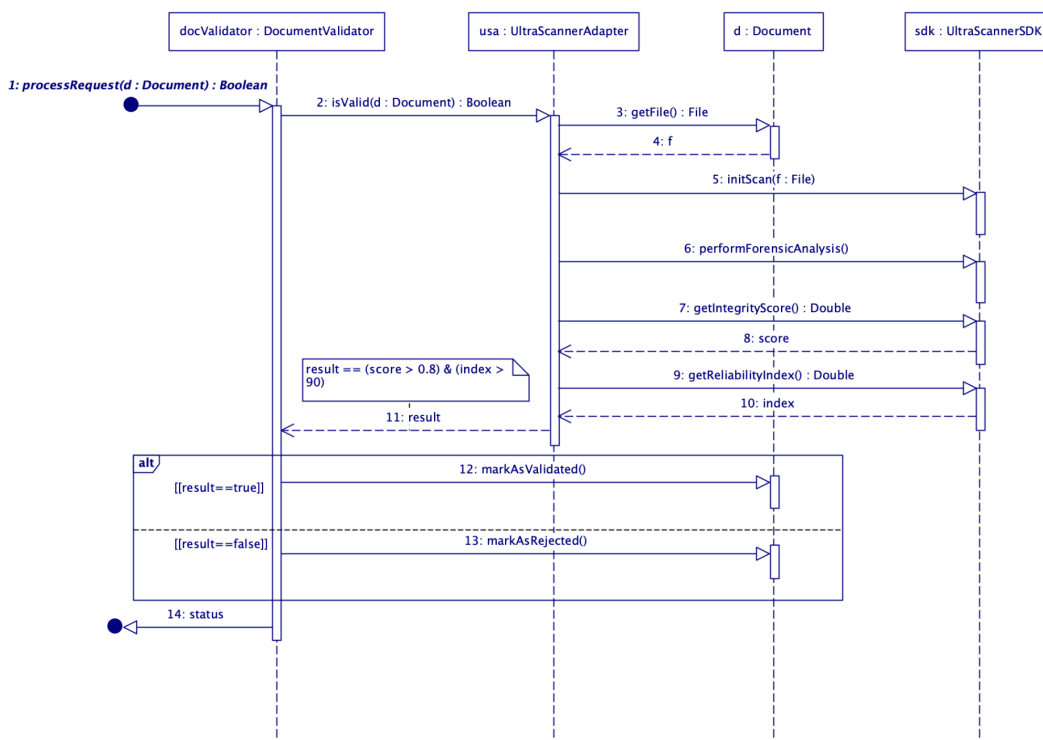
Solución

Pregunta 3.1. Se debe usar el patrón **Adapter** para resolver la integración de la librería externa en el sistema. La situación es que el SDK externo y el controlador no encajan. Mientras que el sistema espera un booleano, la librería externa requiere de varios pasos técnicos y configuraciones manuales. Al introducir una clase que funcione como adaptador, se envuelve toda esa complejidad. Esta clase se encarga de coordinar internamente el escaneo y el análisis para poder traducir esos datos crudos en la respuesta simple que el controlador necesita esperar.

Pregunta 3.2. El diagrama de clases es el siguiente:



Pregunta 3.3. El diagrama de secuencia es el siguiente:



Pregunta 4 (30 %)

La constructora *Percón* necesita un software para generar presupuestos automáticos de proyectos, no se fía de utilizar ninguna IA. El coste final de un proyecto (*Project*) se rige por una fórmula fija: **$totalBudget = (executionCost + materialCost) * (1 + profitMargin)$** . Para realizar este cálculo es necesario tener en cuenta los dos tipos de proyectos principales que existen: Edificación (*HousingProject*) y Obra Civil (*CivilEngineeringProject*).

- En la Edificación, el *executionCost* se calcula multiplicando los metros cuadrados por una tasa fija de vivienda.
- En la Obra Civil, el *executionCost* se calcula mediante un canon gubernamental por kilómetro de superficie.

Por otro lado, el proyecto siempre requiere un Lote de Materiales (*MaterialBatch*). El cálculo del coste de los materiales también sigue un proceso fijo: **$totalMaterialCost = marketPrice + logisticsSurcharge$** .

- Si el lote es Standard, *logisticsSurcharge* es siempre cero.
- Si el lote es Special (Materiales Peligrosos o Pesados), el *logisticsSurcharge* incluye una tasa de transporte especial y un seguro de riesgo.

Se pide:

Pregunta 4.1 (2.5 %). Identifica qué patrón es adecuado para garantizar que el algoritmo de cálculo del presupuesto no sea modificado por las subclases, pero permitiendo que estas definan los costes específicos. Menciona en qué parte se aplica en concreto.

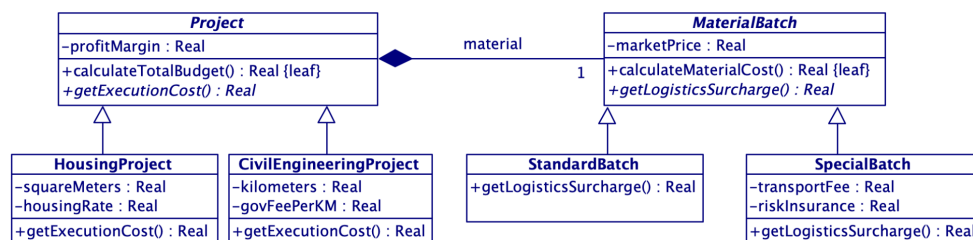
Pregunta 4.2 (12.5 %). Muestra el diagrama de clases UML completo. Es obligatorio que el diagrama refleje qué métodos son los que definen el esqueleto del algoritmo

Pregunta 4.3 (15 %). Diagrama de secuencia de la operación *Project::calculateTotalBudget()*: *Float* cuando se solicita para un proyecto de Edificación que utiliza un lote de Materiales Especiales.

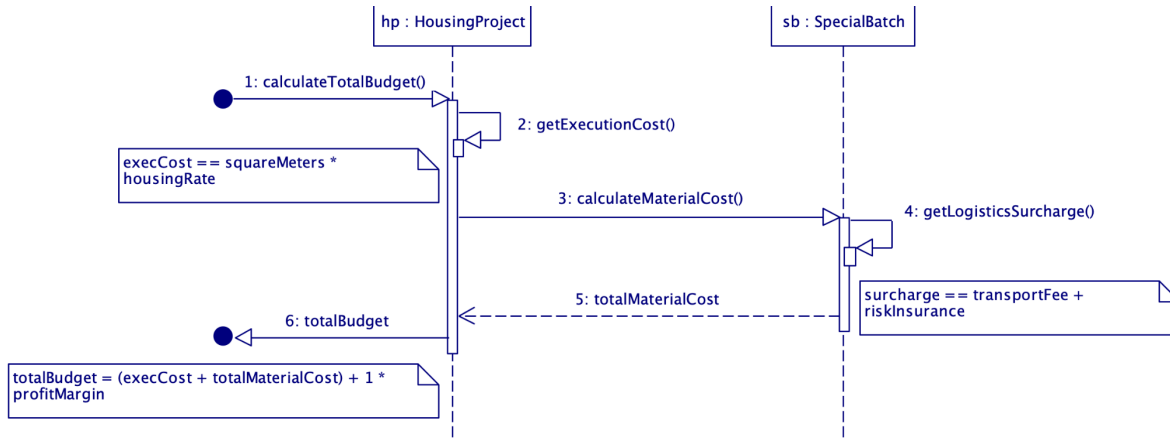
Solución

Pregunta 4.1. Es necesario aplicar dos veces el **método plantilla**. En la clase *Project*, en la fórmula de beneficio corporativo, el método *calculateTotalBudget()* será final. Por otro lado, en la clase *MaterialBatch*, para la lógica de que el coste de material siempre es la base más un recargo, independientemente de qué tipo de recargo sea, el método *getTotalMaterialCost()* será final. Esas serían las dos partes en las que se aplica el método plantilla. La aplicación es doble porque se dispone de dos fórmulas fijas que dependen de detalles específicos que varían según el tipo de elemento. El primer método plantilla termina llamando al segundo método plantilla para obtener sus datos.

Pregunta 4.2. El diagrama de clases sería el siguiente:



Pregunta 4.3. El diagrama de secuencia sería



Criterios de evaluación

La nota final de la PEC se calculará a partir de las notas obtenidas en cada una de las preguntas individuales, con el peso indicado en el enunciado. En concreto, este es el criterio:

- Pregunta 1: 20 %
- Pregunta 2: 20 %
- Pregunta 3: 30 %
- Pregunta 4: 30 %

Esta PEC corresponde a un **45 % de la nota de Evaluación Continua de la asignatura**.



RECUERDA: Para acogerte a la evaluación continua de la asignatura, deberás entregar todas las PECs y PRAs y **superar cada una de ellas con al menos 3 puntos**.

Tal como se explica en el plan docente de la asignatura, esta actividad debe resolverse de forma individual. Por otro lado, no está permitido el uso de herramientas de inteligencia artificial. Recuerda que debes incluir el texto indicado en la sección “Compromiso de autorresponsabilidad” de tu PEC. En caso de detectar copias o uso de herramientas de IA, se penalizará la actividad con un 0 % como nota.

Formato de entrega

- El documento que entregues con tu propuesta de solución para esta PEC **debe seguir el formato que encontrarás en el aula**. Podrás descargar el fichero (en formato TXT) y elaborar tu solución en cualquier editor de textos, pero recuerda que la estructura debe ser la establecida en dicho fichero.
- Deberás entregar un **único documento** (formato pdf o docx) con tu propuesta de solución para las preguntas del enunciado.
- Recuerda que debes producir diagramas **conforme a la especificación UML** y **ser consistente** en el estilo de diagramas que entregues en todos los ejercicios.
- Todos los documentos se tienen que entregar en el aula virtual antes de las 23:59 horas del **12 de mayo de 2026**.
- **No se aceptarán entregas fuera de plazo**.

Propiedad intelectual y plagio

La Normativa académica de la UOC dispone que el proceso de evaluación se cimenta en el trabajo personal del estudiante y presupone la autenticidad de la autoría y la originalidad de los ejercicios realizados.

La ausencia de originalidad en la autoría o el mal uso de las condiciones en las que se realiza la evaluación de la asignatura, es una infracción que puede tener consecuencias académicas graves.

Te recomendamos leer esta [guía sobre el plagio académico y como evitarlo](#). Recuerda que siempre que reutilices contenido de terceros, debes citar la fuente. Puedes leer también esta [guía para saber cómo citar correctamente](#).

El estudiante será calificado con un suspenso (0 %) si se detecta falta de originalidad en la autoría, sea porque haya utilizado material o dispositivos no autorizados, sea porque ha copiado textualmente de internet, o ha copiado apuntes, de otras PECs, de materiales, manuales o artículos (sin la cita correspondiente) o de otro estudiante, o por cualquier otra conducta irregular.